Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

4. Giao tiếp đồng bộ — REST, gRPC & Resilience . . . 2
4.1. Giao tiếp đồng bộ — khi nào nên, khi nào tránh? . . . 3
4.1.1. Request-Response: mô hình quen thuộc . . . 3
4.1.2. Ưu và nhược điểm . . . 3
4.1.3. Design-time Coupling vs Runtime Coupling . . . 4
4.1.4. Khi nào dùng sync? . . . 5
4.1.5. Interaction Styles — phân loại kiểu tương tác . . . 5
4.1.6. REST vs gRPC — hai lựa chọn cho sync . . . 5
4.2. OpenFeign — Declarative REST Client . . . 7
4.2.1. Vấn đề: gọi service khác không đơn giản như gọi hàm . . . 7
4.2.2. OpenFeign: gọi REST như gọi hàm . . . 7
4.2.3. Cách hoạt động . . . 8
4.3. Service Discovery & Load Balancing . . . 8
4.3.1. Vấn đề: URL cứng trong distributed system . . . 9
4.3.2. Service Discovery patterns . . . 9
4.3.3. Netflix Eureka trong KBM . . . 10
4.4. Resilience Patterns — sống sót trong Distributed System . . . 10
4.4.1. Resilience Metrics — đo lường độ tin cậy . . . 10
4.4.2. Tại sao cần resilience? . . . 11
4.4.3. Bốn resilience pattern cốt lõi . . . 12
4.4.4. Triển khai thực tế với Resilience4j — mã ví dụ cho LMS . . . 15
4.5. Case study: KBM . . . 18
4.5.1. Bài toán . . . 18
4.5.2. Hiện trạng: Strategy Pattern + OpenFeign . . . 19
4.5.3. Phân tích các vấn đề . . . 19
4.5.4. Lộ trình chuyển đổi đề xuất . . . 20
4.5.5. Tích hợp ngoài: QLĐT, GitHub Classroom và Network Practice . . . 21
4.6. Tổng kết . . . 22
4.7. Đọc thêm . . . 23
Tài liệu tham khảo . . . 23

4

CHƯƠNG 4

Giao tiếp đồng bộ — REST, gRPC & Resilience

> "Smart endpoints and dumb pipes."
> — James Lewis & Martin Fowler, *Microservices* (2014)

MỤC TIÊU HỌC TẬP

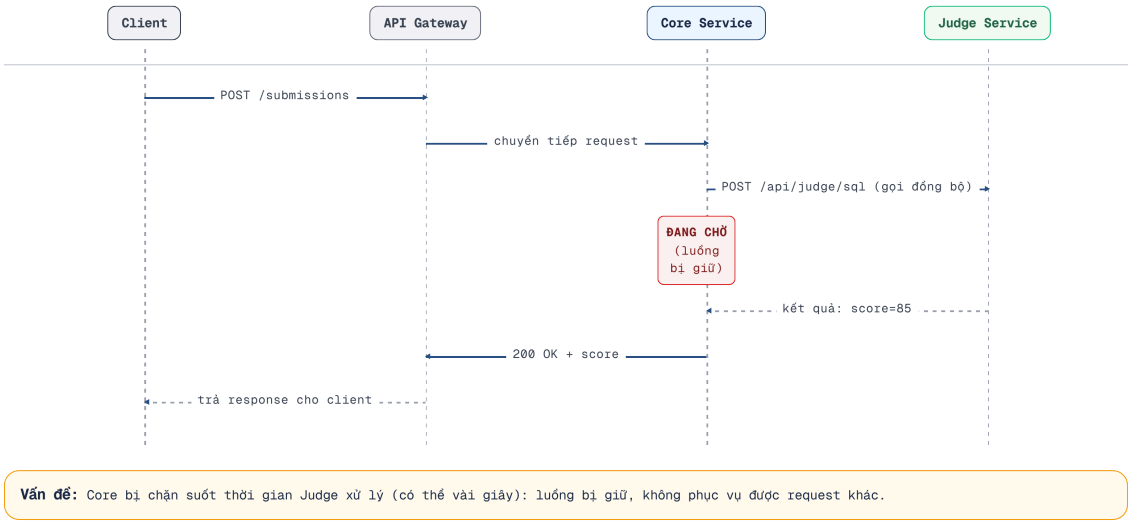
- Hiểu các kiểu tương tác (interaction styles) trong microservices
- So sánh REST và gRPC — khi nào dùng cái nào
- Sử dụng OpenFeign để thực hiện các cuộc gọi liên dịch vụ (service-to-service) theo mô hình khai báo (declarative)
- Nắm vững Service Discovery và Load Balancing với Eureka
- Áp dụng các resilience pattern: Circuit Breaker, Retry, Timeout, Bulkhead
- Phân tích bài toán dispatch SQL và tích hợp ngoài trong KBM

4.1. Giao tiếp đồng bộ — khi nào nên, khi nào tránh?

Trước khi phân tích chi tiết, hãy hình dung giao tiếp đồng bộ giống như việc sinh viên lên thẳng Phòng Đào tạo để nộp đơn và phải đứng đợi cho đến khi chuyên viên xử lý xong mới được ra về. Sự chờ đợi này đảm bảo tính chắc chắn, nhưng lại gây tắc nghẽn khi có quá đông người.

4.1.1. Request-Response: mô hình quen thuộc

Giao tiếp đồng bộ (*synchronous communication*) là mô hình tương tác cơ bản và quen thuộc nhất đối với hầu hết các nhà phát triển phần mềm. Trong mô hình này, khi dịch vụ A cần thông tin hoặc yêu cầu xử lý từ dịch vụ B, nó sẽ gửi một yêu cầu (request) qua mạng, bị chặn luồng (block) và phải **chờ đợi** cho đến khi nhận được phản hồi (response) trả về. Chỉ khi có kết quả từ dịch vụ B, dịch vụ A mới tiếp tục thực thi các logic nghiệp vụ tiếp theo của nó. HTTP, thường theo phong cách REST, hiện là cách phổ biến nhất để hiện thực hóa mô hình này nhờ tính đơn giản, dễ tích hợp và khả năng khai thác hệ sinh thái công cụ phong phú. Tuy nhiên, đằng sau sự đơn giản đó là những chi phí ngầm về độ trễ và sự phụ thuộc chặt chẽ về mặt thời gian giữa các thành phần hệ thống.



Hình 4.1: *Giao tiếp đồng bộ — Core Service bị chặn (blocked) trong khi chờ Judge phản hồi*

4.1.2. Ưu và nhược điểm

Bảng dưới đây tổng hợp các ưu và nhược điểm cốt lõi của giao tiếp đồng bộ:

Bảng 4.1: *Ưu và nhược điểm của giao tiếp đồng bộ*

Khía cạnh	Ưu điểm	Nhược điểm
Đơn giản	Mô hình request-response quen thuộc, dễ gỡ lỗi (debug)	Bên gọi (caller) bị chặn (**block**) hoặc đình trệ cho đến khi nhận được phản hồi (response)
Nhất quán	Nhận kết quả trực tiếp khi có phản hồi; timeout có thể để kết quả nghiệp vụ chưa xác định	**Temporal coupling**: cả hai dịch vụ phải khả dụng (available) cùng lúc
Tooling	HTTP tooling phong phú (Postman, curl, browser)	**Cascading failure**: service B ngừng hoạt động → A cũng ngừng hoạt động
Tracing	Ngữ cảnh truy vết (`traceparent` / `traceId`) truyền tuần tự qua HTTP header theo luồng lời gọi trực tiếp (xem Ch.11)	**Độ trễ cộng dồn**: A→B→C = tổng độ trễ

Richardson phân tích trong [2a, Ch.3] rằng giao tiếp đồng bộ làm giảm tính sẵn sàng: dịch vụ gọi không phản hồi được nếu dịch vụ được gọi ngừng hoạt động. Hiện tượng này thường được gọi là **temporal coupling** (cả hai dịch vụ phải sẵn sàng cùng lúc); bản thứ hai của sách gọi là **runtime coupling** [2b, Ch.4]. Đây là lý do kiến trúc microservices thường ưu tiên giao tiếp bất đồng bộ (async) cho các luồng xử lý không yêu cầu phản hồi tức thời.

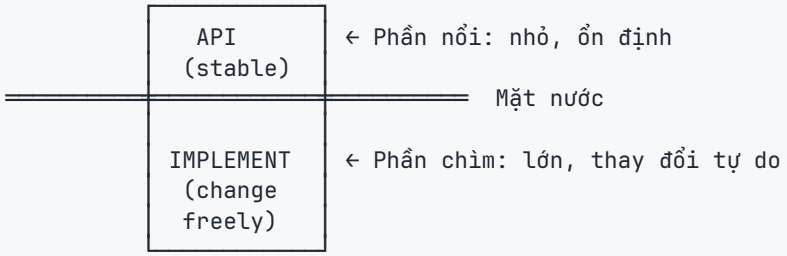
4.1.3. Design-time Coupling vs Runtime Coupling

Richardson trong phiên bản thứ hai [2b, Ch.4] phân biệt bốn loại coupling: design-time, build-time, runtime và infrastructure. Khi chọn kiểu giao tiếp, hai loại cần tách bạch nhất là design-time và runtime; hiểu sai sự khác biệt giữa chúng dễ dẫn đến thiết kế microservices kém:

Bảng 4.2: *Hai loại coupling trong microservices*

	Design-time Coupling	Runtime Coupling
Định nghĩa	Thay đổi dịch vụ A → **phải** thay đổi dịch vụ B	Dịch vụ A **cần** B đang chạy để xử lý yêu cầu (request)
Khi nào xảy ra	Khi thiết kế và sửa mã	Khi hệ thống đang chạy (runtime)
Ảnh hưởng	Giảm tính tự chủ của nhóm (team autonomy), tăng sự phụ thuộc/phối hợp	Giảm tính sẵn sàng (availability), tăng độ trễ (latency)
Ví dụ KBM	Core Service đổi DTO câu hỏi → Academic Service phải sửa phần đọc	Core Service gọi sync Judge → Judge ngừng hoạt động → Core trả lỗi
Giải pháp	API versioning (§3.2), backward compatible changes	Async messaging (Ch.5), Circuit Breaker (§4.4)

Iceberg Principle — Richardson gọi đây là nguyên tắc nền tảng cho loose coupling [2b, Ch.4]:



API (phần nổi) phải nhỏ nhất có thể — chỉ bộc lộ (expose) những gì thành phần gọi/sử dụng dịch vụ (consumer) thực sự cần. Implementation (phần chìm) có thể lớn và thay đổi tự do mà không làm ảnh hưởng đến các consumer này. **Đây cũng là lý do dùng DTO pattern** (§3.4) thay vì trả entity trực tiếp — entity là "phần chìm" không nên lộ ra.

Nghịch lý DRY và Coupling

Trong monolith, DRY (Don't Repeat Yourself) gần như luôn đúng. Nhưng trong microservices, DRY có thể tạo coupling không mong muốn. Ví dụ: Lớp `ScoreCalculator` (quy tắc tính điểm) được sử dụng chung thông qua một thư viện chia sẻ (shared library). Khi đó, mỗi lần cập nhật quy tắc nghiệp vụ (business rule), tất cả các dịch vụ phụ thuộc vào thư viện này đều phải được nâng cấp đồng loạt (lock-step deployment). Đôi khi **duplication có chủ đích** tốt hơn coupling — đặc biệt khi logic ở mỗi context có lý do để đi theo hướng riêng. Richardson trong [2b, Ch.4] khuyên: chỉ áp dụng nguyên tắc DRY khi việc các bản triển khai khác nhau thực sự gây ra lỗi (**bug**), không nên áp dụng một cách khiên cưỡng trong mọi trường hợp.

4.1.4. Khi nào dùng sync?

Giao tiếp đồng bộ phù hợp khi:

- **Cần phản hồi ngay**: query data (GET), validation, authentication
- **Flow đơn giản**: chuỗi gọi dịch vụ ngắn (A→B), không phải chuỗi dài (A→B→C→D)
- **Các thao tác nghiêng về truy xuất dữ liệu (Read-heavy operations)**: lấy thông tin, không phải thay đổi trạng thái phức tạp

Không phù hợp khi:

- **Các tác vụ thực thi kéo dài (Long-running operations)**: chấm bài SQL mất 5–30 giây
- **Tác vụ gửi-và-quên (Fire-and-forget)**: gửi thông báo (notification), ghi nhận nhật ký hệ thống (log event)
- **Giao dịch liên dịch vụ (Cross-service transactions)**: cần saga pattern (Chương 6)

Một nguyên tắc kinh nghiệm đơn giản: nếu bên gọi (caller) *phải biết* kết quả để tiếp tục xử lý → chọn giao tiếp đồng bộ; nếu bên gọi chỉ cần *kích hoạt* hành động và không cần đợi kết quả → chọn giao tiếp bất đồng bộ (Chương 5). Trong hệ thống KBM, việc truy vấn trạng thái hoạt động (health query) của Judge sử dụng cơ chế đồng bộ, nhưng việc gửi yêu cầu chấm bài (submit) lại sử dụng cơ chế bất đồng bộ thông qua Kafka — phản ánh hai nhu cầu hoàn toàn khác biệt.

4.1.5. Interaction Styles — phân loại kiểu tương tác

Richardson phân loại các kiểu giao tiếp trong microservices dựa trên hai không gian chính: số lượng bên tiếp nhận (one-to-one so với one-to-many) và tính đồng bộ (synchronous so với asynchronous) [2a, Ch.3]. Trong khi giao tiếp đồng bộ thường bị giới hạn ở kiểu một-một (như Request/Response quen thuộc), thì giao tiếp bất đồng bộ cung cấp tính linh hoạt cao hơn. Nó không chỉ hỗ trợ gọi một-một (như thông báo một chiều hay request/response bất đồng bộ), mà còn đặc biệt hiệu quả ở mô hình một-nhiều (như Publish/Subscribe hay Publish/Async responses) nơi một thông điệp có thể kích hoạt hàng loạt dịch vụ cùng lúc.

Hầu hết service dùng kết hợp nhiều kiểu. Ví dụ: KBM Core Service dùng *request/response* (Feign) để truy vấn hệ thống chấm điểm (Judge), *publish/subscribe* (Kafka) để gửi bài nộp (submission), và *one-way notification* (WebSocket) để đẩy/trả kết quả.

4.1.6. REST vs gRPC — hai lựa chọn cho sync

Khi quyết định sử dụng giao tiếp đồng bộ, REST (dựa trên HTTP và JSON) thường là lựa chọn mặc định. Bên cạnh REST, **gRPC** — một framework RPC mã nguồn mở hiệu năng cao do Google phát triển — là giải pháp phổ biến cho giao tiếp nội bộ giữa các microservice đòi hỏi tốc độ cao, độ trễ thấp và băng thông được tối ưu bằng định dạng nhị phân. Bảng sau so sánh chi tiết hai công nghệ này:

Bảng 4.3: *REST vs gRPC — so sánh chi tiết*

Tiêu chí	REST (HTTP/JSON)	gRPC (HTTP/2 + Protobuf)
Format	JSON (text, human-readable)	Protocol Buffers (binary, compact)
Performance	Chậm hơn (text parsing)	Thường nhanh hơn đáng kể, nhất là với khối lượng dữ liệu (payload) lớn / cần nhiều lượt đi-về (round-trip) (binary + HTTP/2 multiplexing); mức cải thiện tùy thuộc vào tải hệ thống (workload)
Contract	OpenAPI spec (optional)	`.proto` file (bắt buộc)
Code gen	Optional (OpenAPI generators)	Built-in (protoc generates client/server)

Tiêu chí	REST (HTTP/JSON)	gRPC (HTTP/2 + Protobuf)
Browser support	Native	Cần gRPC-Web proxy
Streaming	Không native (phải dùng SSE/WebSocket)	Bi-directional streaming native
Debugging	Dễ dàng (hỗ trợ curl, Postman, trình duyệt)	Phức tạp (cần gRPC tools)

gRPC Architecture — Tại sao nhanh hơn? gRPC khác REST ở ba yếu tố kiến trúc; hai yếu tố đầu giải thích vì sao nó thường nhanh hơn, yếu tố thứ ba mở thêm kiểu giao tiếp:

1. **HTTP/2 multiplexing** — Nhiều yêu cầu (request) chạy đồng thời trên cùng một kết nối TCP, không phải chờ nhau như trên HTTP/1.1. Quan trọng khi dịch vụ A gọi dịch vụ B hàng trăm lần/giây.
2. **Protocol Buffers** — Schema-first, binary encoding. Payload thường nhỏ hơn JSON đáng kể — tỉ lệ phụ thuộc dữ liệu: chênh nhiều với payload nhiều số/enum, thu hẹp với payload chủ yếu là chuỗi văn bản hoặc khi JSON đã nén gzip. Schema evolution built-in (backward/forward compatible) — `.proto` file *là* contract.
3. **Four communication patterns**: Unary (request-response, như REST), Server streaming (server gửi stream), Client streaming (client gửi stream), **Bi-directional streaming** (cả hai gửi stream đồng thời). **Lưu ý quan trọng:** Bi-directional streaming rất hiệu quả cho backend-to-backend. Trình duyệt (thông qua gRPC-Web) có hỗ trợ Server-Streaming, nhưng chưa hỗ trợ Bi-directional streaming. Dù có thể dùng Server-streaming để trả kết quả (ví dụ: stream kết quả từng test case cho trình duyệt của sinh viên), WebSocket hoặc SSE (Server-Sent Events) vẫn là chuẩn phổ biến hơn cho real-time frontend.

Lưu ý về Load Balancing: Vì gRPC duy trì kết nối HTTP/2 dài hạn (persistent connection), các Load Balancer L4 thông thường (phân phối theo kết nối TCP) sẽ chia tải bị lệch. Để cân bằng tải trên từng request, hãy dùng L7 Load Balancer (như Envoy, HAProxy) hoặc Service Mesh; một lựa chọn khác là client-side load balancing qua service discovery khi client nội bộ đáng tin cậy. L4 vẫn dùng được cho các thiết lập đơn giản, nhưng phải chấp nhận rủi ro lệch tải theo kết nối.

Khi nào dùng gRPC thay REST?

- **Giao tiếp nội bộ (Internal service-to-service)**: Khi hiệu năng là ưu tiên hàng đầu và không yêu cầu tương tác trực tiếp từ trình duyệt → chọn gRPC
- **API công khai / Giao diện người dùng (Frontend)**: Khi cần sự hỗ trợ từ trình duyệt và dễ dàng gỡ lỗi thủ công → chọn REST
- **Truyền phát dữ liệu (Streaming)**: Khi yêu cầu giao tiếp hai chiều thời gian thực (real-time bi-directional) → chọn gRPC (hoặc WebSocket)
- **Môi trường Polyglot**: đội ngũ phát triển sử dụng nhiều ngôn ngữ lập trình khác nhau → gRPC (code gen đa ngôn ngữ)

KBM không còn là hệ thống REST thuần

LMS core của KBM vẫn ưu tiên REST/JSON vì các trình duyệt (browser), bảng điều khiển (dashboard) và bộ công cụ OpenAPI yêu cầu khả năng gỡ lỗi (debug) dễ dàng. Nhưng Network Lab chủ động dùng TCP/UDP/RMI/SOAP/REST/gRPC để sinh viên thấy được sự khác biệt giữa các giao thức (protocol) trong cùng một lĩnh vực (domain) bài tập. Vì vậy, chuyển sang gRPC không phải mục tiêu "đổi toàn bộ REST"; cách đúng hơn là **chọn protocol theo boundary**: Sử dụng REST cho các API hướng người dùng (client-facing), gRPC cho các giao tiếp nội bộ đòi hỏi thông lượng cao hoặc truyền phát dữ liệu (high-throughput/streaming) khi cần thiết, và các

hàng đợi lô/công việc (batch/job queue) cho các tác vụ chấm điểm (judge) kéo dài hoặc mang tính trạng thái (stateful).

4.2. OpenFeign — Declarative REST Client

Thay vì bắt sinh viên tự cầm hồ sơ gõ cửa từng phòng ban, hệ thống một cửa OpenFeign tự động chuẩn hóa mọi thủ tục giao tiếp. Cách tiếp cận này giúp mã gọi dịch vụ nội bộ gọn hơn và ít lặp lại hơn.

4.2.1. Vấn đề: gọi service khác không đơn giản như gọi hàm

Trong kiến trúc nguyên khối (monolith), gọi module khác chỉ là một dòng mã: `judgeService.execute(request)`. Trong microservices, cùng logic đó trở thành: xây dựng HTTP request, tuần tự hóa dữ liệu (serialize payload), gắn header (JWT token), gửi qua mạng, xử lý mã phản hồi (response code), giải tuần tự hóa kết quả (deserialize result) và xử lý lỗi hoặc quá hạn (handle timeout/error). Và điều nguy hiểm nhất: khi gọi hàm nội bộ, nếu có lỗi, chương trình sẽ ném exception ngay lập tức. Nhưng khi gọi qua mạng, hệ thống có thể bị đình trệ vô thời hạn mà không hề ném ra exception nào. Trong hệ thống phân tán, *một phản hồi chậm (slow response) còn nguy hiểm hơn rất nhiều so với một lỗi trả về ngay lập tức (fail-fast)*.

4.2.2. OpenFeign: gọi REST như gọi hàm

Để giải quyết sự cồng kềnh của các HTTP client truyền thống, hệ sinh thái Spring cung cấp **OpenFeign** (Spring Cloud OpenFeign). Công cụ này mang đến một cách tiếp cận mang tính khai báo (declarative), cho phép các lập trình viên chỉ cần định nghĩa các lời gọi API dưới dạng một Java interface thông thường. Framework tự sinh mã thực thi (implementation) lúc runtime cho các tác vụ như thiết lập kết nối, quản lý header và chuyển đổi dữ liệu. Từ Spring Cloud 2022.0.0, OpenFeign chỉ còn được sửa lỗi; tài liệu chính thức gợi ý mã mới dùng Spring HTTP Service Clients (`@HttpExchange`). Sách vẫn dùng OpenFeign vì KBM đang dùng, và cơ chế client khai báo ở hai công cụ là như nhau.

Listing 4.1: *So sánh RestTemplate (viết dài dòng) và OpenFeign (dạng khai báo)*

```
// Cách thủ công: dựng HTTP request và xử lý response
RestTemplate restTemplate = new RestTemplate();
HttpHeaders headers = new HttpHeaders();
headers.setBearerAuth(jwtToken);
HttpEntity<JudgeRequest> entity = new HttpEntity<>(request, headers);
ResponseEntity<JudgeResult> response = restTemplate.exchange(
    url, HttpMethod.POST, entity, JudgeResult.class
);

// Cách khai báo: mô tả API bằng Feign interface
@FeignClient(name = "kbm-judge-sql")
public interface JudgeClient {
    @PostMapping("/api/judge/sql")
    // Gọi như hàm bình thường!
    JudgeResult submitSql(@RequestBody JudgeRequest request);
}
```

Mô hình khai báo của OpenFeign giúp mã nguồn gọn hơn hẳn lối viết thủ công bằng `RestTemplate` và loại bỏ nhiều lỗi tiềm ẩn do cấu hình thiếu sót. Tuy nhiên, sự trừu tượng hóa này cũng có thể vô tình che khuất một số thiết lập cấu hình mạng quan trọng ở tầng dưới, đặc biệt là cách quản lý

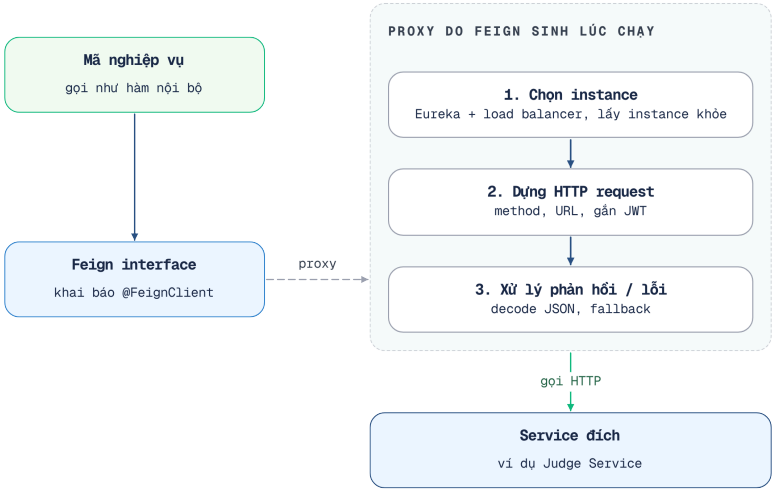
vòng đời của các kết nối TCP mà nếu không chú ý sẽ nhanh chóng trở thành nút thắt cổ chai về hiệu năng của toàn hệ thống.

Cấu hình Connection Pooling cho OpenFeign

Mặc định, khi classpath không có HTTP client khác, OpenFeign sử dụng `HttpURLConnection` của Java — không cung cấp một connection pool được quản lý đúng nghĩa (dù có keep-alive cơ bản, nó không gộp và tái sử dụng kết nối hiệu quả dưới tải cao), dẫn đến suy giảm hiệu năng do chi phí thiết lập kết nối lặp lại. Trên production, đội ngũ phát triển nên cấu hình OpenFeign sử dụng `Apache HttpClient 5` hoặc `OkHttp` để khai thác cơ chế quản lý connection pool, giúp giảm thiểu độ trễ mạng.

4.2.3. Cách hoạt động

Hình dưới đây cho thấy các bước Feign làm thay lập trình viên.




```java
@FeignClient(name = "kbm-judge-sql")
public interface JudgeClient {
  @PostMapping("/api/judge/sql")
  JudgeResult submitSql(@RequestBody JudgeRequest req);
}
```

Hình 4.2: Cách Feign proxy tự động xử lý HTTP request, JWT, load balancing

Feign tự động xử lý: serialization/deserialization JSON sang Java, header injection (ví dụ JWT token qua `RequestInterceptor`), service discovery (tra cứu Eureka thay vì gán cứng URL), và error decoding (chuyển đổi response code thành exception).

4.3. Service Discovery & Load Balancing

Khi quy mô hệ thống trường học mở rộng, việc ghi nhớ vị trí chính xác của từng phòng ban trở nên bất khả thi. Service Discovery hoạt động như một bản đồ điện tử động của toàn trường, giúp các ứng dụng luôn tìm thấy nhau mà không cần cấu hình cứng.

4.3.1. Vấn đề: URL cứng trong distributed system

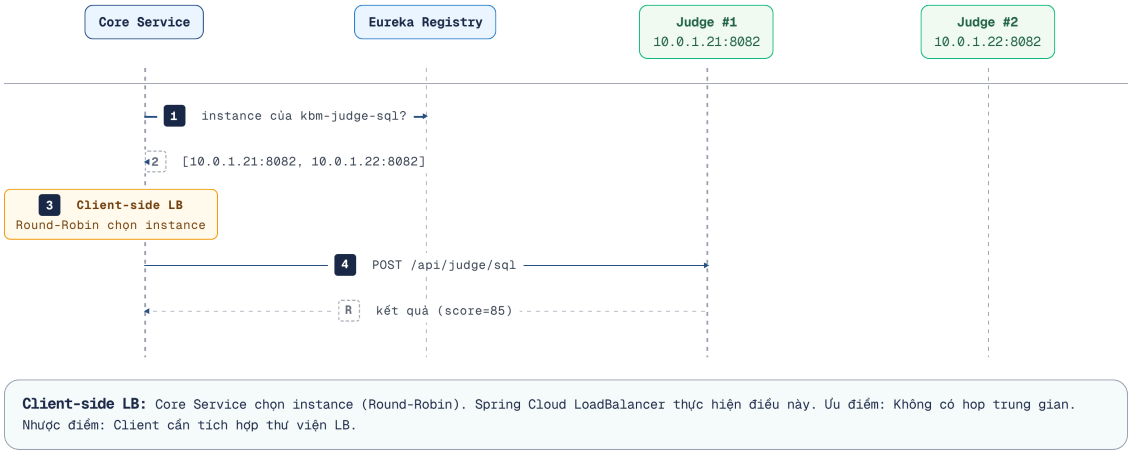
Trong monolith, gọi module khác là gọi hàm — không cần biết "module ở đâu". Trong microservices, mỗi dịch vụ là một tiến trình (process) độc lập, chạy trên các địa chỉ và cổng mạng (host:port) khác nhau. Câu hỏi: **dịch vụ A tìm dịch vụ B ở đâu?**

Cách đơn giản nhất: gán cứng (hardcode) URL trong cấu hình. Nhưng khi dịch vụ mở rộng quy mô (scale) thành 3 instance, hệ thống nên sử dụng URL nào? Khi dịch vụ được triển khai (deploy) sang container mới, địa chỉ IP thay đổi — cơ chế nào sẽ chịu trách nhiệm cập nhật?

4.3.2. Service Discovery patterns

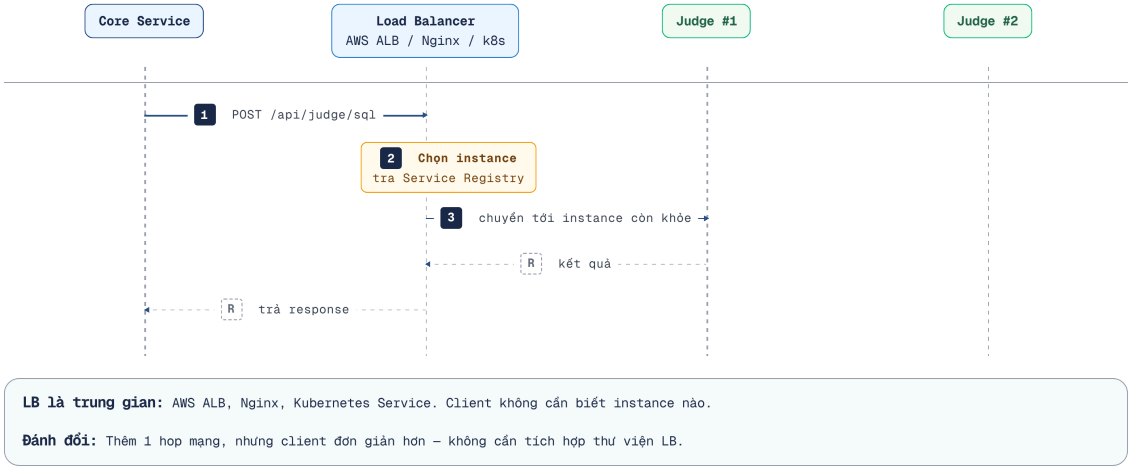
Có hai pattern chính [2a, Ch.3]:

Client-side discovery — Client (dịch vụ gọi) tự truy vấn (query) service registry để lấy danh sách instance, sau đó tự phân tải (load balance):



Hình 4.3: Client-side discovery — service tự truy vấn Eureka và tự cân bằng tải

Server-side discovery — Load balancer (hoặc API Gateway) hoạt động như một thành phần trung gian, client chỉ cần biết URL của load balancer:



Hình 4.4: Server-side discovery — load balancer điều hướng request

4.3.3. Netflix Eureka trong KBM

KBM sử dụng **Netflix Eureka** cho cụm các service Java/Spring — client-side discovery pattern:

Khi microservice khởi động, nó đăng ký tại Eureka server với tên (`spring.application.name`) và địa chỉ. Mặc định, Eureka client chỉ gửi nhịp tim định kỳ (mỗi 30 giây) để báo tiến trình còn sống; muốn đồng bộ trạng thái kiểm tra sức khỏe sâu của Spring Boot Actuator (như kết nối CSDL, ổ đĩa) lên registry thì cần bật tường minh `eureka.client.healthcheck.enabled=true` (xem §11.6). Gateway và các dịch vụ khác sẽ truy vấn (query) Eureka để tìm các instance đang hoạt động. Trong cấu hình (config) Gateway: `uri: lb://kbm-core` nghĩa là tra cứu Eureka tìm tất cả instance đăng ký dưới tên `kbm-core`, rồi phân tải (load balance) lưu lượng giữa chúng. Client giữ bản sao danh sách instance trong bộ nhớ và làm mới định kỳ (mặc định 30 giây), nên khi registry tạm ngừng, các lời gọi đang có vẫn chạy theo danh sách đã lưu; điều này không bảo đảm cho client mới chưa có bản sao, hoặc instance đã đổi địa chỉ.

Service naming không nhất quán

Trong snapshot cũ của KBM, `kbm-core` và `kbm-academic` từng dùng `spring.application.name = "app"` — vi phạm nguyên tắc unique service identity [2a]. Khi Eureka nhận hai service cùng tên, nó coi chúng là các instance của *cùng một dịch vụ* → dẫn đến định tuyến sai. Đây là hậu quả của việc `kbm-academic` tách ra từ Core nhưng không đổi service name. **Lộ trình chuyển đổi (Migration path)**: đổi thành `kbm-core` và `kbm-academic`, cập nhật Eureka config và Gateway routes.

Cần lưu ý rằng Eureka là một *application-level* service discovery — service tự đăng ký và tự tra cứu trong code (qua Spring Cloud). Nếu hệ thống được triển khai trên **Kubernetes**, phần lớn vai trò này đã có sẵn ở tầng hạ tầng: mỗi service là một `Service` object, K8s sẽ đảm nhiệm việc đăng ký, DNS nội bộ (`http://kbm-core`) và load balancing — thường *không cần* Eureka nữa. Nói gọn: dùng Eureka khi chạy ngoài K8s (VM, bare-metal, Spring Cloud thuần); cân nhắc bỏ Eureka khi đã lên K8s để tránh trùng lặp hai lớp discovery. KBM hiện dùng Eureka vì cụm Java/Spring chạy bằng Docker Compose, chưa lên K8s; lộ trình lên k3s (Chương 12) sẽ mở ra lựa chọn chuyển sang discovery K8s-native.

4.4. Resilience Patterns — sống sót trong Distributed System

Môi trường mạng không đáng tin cậy cũng giống như việc mất điện hay rớt mạng cục bộ trong ký túc xá. Các resilience pattern giúp KBM không sụp đổ dây chuyền khi điều đó xảy ra.

4.4.1. Resilience Metrics — đo lường độ tin cậy

Để đánh giá độ tin cậy của một hệ thống có thể khôi phục (repairable system), giới kỹ thuật thường sử dụng bốn thước đo cốt lõi gắn kết chặt chẽ với nhau. Khoảng thời gian trung bình hệ thống vận hành trơn tru kể từ khi sẵn sàng cho đến lúc phát sinh sự cố tiếp theo được gọi là **MTTF (Mean Time To Failure)**. Khi sự cố xảy ra, tốc độ khắc phục được đo bằng **MTTR (Mean Time To Recovery)** — khoảng thời gian trung bình từ lúc hệ thống ngưng trệ đến khi khôi phục hoàn toàn. Tổng thời gian của một chu kỳ đầy đủ giữa hai sự cố liên tiếp chính là **MTBF (Mean Time Between Failures)**, với quan hệ `MTBF = MTTF + MTTR` (ví dụ: một service trung bình mỗi tuần gặp sự cố một lần thì MTBF là 168 giờ). Từ ba đại lượng trên, **Availability (Tính sẵn sàng)** được tính bằng tỷ lệ phần trăm thời gian hệ thống thực sự phục vụ người dùng: `Availability = MTTF / (MTTF + MTTR) = MTTF / MTBF`. Ví dụ: với MTBF 168 giờ và MTTR 1 giờ, Availability = 167 / 168 ≈ 99.4%.

Thuật ngữ "nines" (các số 9) thường được sử dụng trong ngành công nghiệp phần mềm để mô tả nhanh các mức độ khả dụng mục tiêu của một hệ thống. Mỗi "số 9" được thêm vào đại diện cho những thách thức kỹ thuật lớn, đi kèm với sự phức tạp và chi phí vận hành cao hơn. Bảng 4.4 minh họa quy đổi từ các tỷ lệ phần trăm này sang thời gian ngừng hoạt động tối đa cho phép, giúp chúng ta hình dung rõ ràng hơn về mức độ khắt khe của từng cấp độ cam kết dịch vụ.

Bảng 4.4: *Các mức "nines" của tính sẵn sàng — downtime cho phép theo từng mục tiêu*

Mục tiêu	Downtime/năm	Downtime/tháng	Phù hợp cho
99% (two nines)	3.65 ngày	7.3 giờ	Internal tools, dev environments
99.9% (three nines)	8.76 giờ	43.8 phút	**KBM** — mục tiêu cho chế độ thi (tính trên khung giờ từng ca thi, xem Ch.11)
99.99% (four nines)	52.6 phút	4.38 phút	E-commerce, banking
99.999% (five nines)	5.26 phút	26.3 giây	Critical infrastructure

Trong thực tế, các mục tiêu khả dụng này định hình trực tiếp quyết định thiết kế và cách vận hành hằng ngày. Để chúng trở thành cơ chế điều tiết rủi ro và nhịp phát hành phần mềm thay vì chỉ là con số lý thuyết, giới kỹ thuật áp dụng khái niệm **ngân sách lỗi** (error budget).

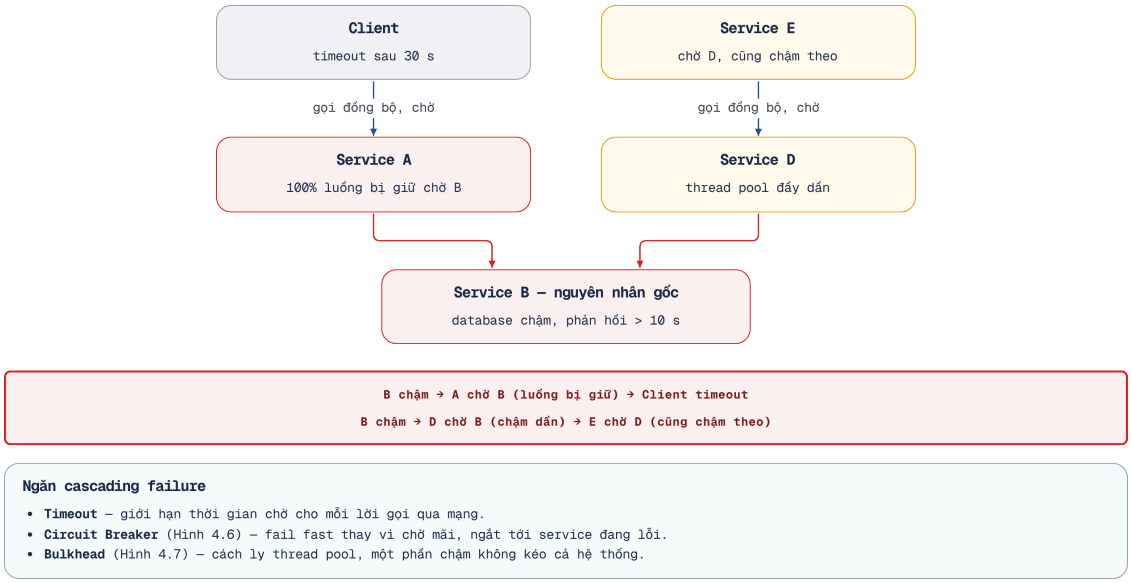
Error Budgets

Google SRE (Site Reliability Engineering) đề xuất: nếu target availability = 99.9%, hệ thống có **error budget = 0.1%** downtime/tháng (43.8 phút). Ngân sách lỗi này sẽ được dùng để đánh đổi cho các hoạt động như: triển khai phiên bản mới (deployments), thử nghiệm tính năng (experimentation) hoặc bảo trì có kiểm soát (controlled maintenance). Khi cạn kiệt ngân sách lỗi, đội ngũ vận hành sẽ đóng băng việc triển khai (freeze deployments) và phải tập trung hoàn toàn vào việc duy trì tính ổn định của hệ thống. Đây là cách biến "availability target" từ khái niệm mơ hồ thành **nguyên tắc vận hành cụ thể** (xem thêm SLI/SLO/SLA ở Ch.11).

4.4.2. Tại sao cần resilience?

Trong kiến trúc nguyên khối (monolith), nếu cơ sở dữ liệu chậm, *toàn bộ* ứng dụng sẽ chậm theo — nhưng ít nhất lỗi được thể hiện rõ ràng. Trong microservices, khi dịch vụ B chậm, dịch vụ A *vẫn tiếp tục chạy* nhưng các luồng xử lý (thread) bị chặn lại để chờ B → các yêu cầu (request) gửi đến A cũng bị chậm lại → client gọi A cũng bị chậm → tạo thành lỗi dây chuyền (**cascading failure**) lan khắp hệ thống [5, Ch.7]. Hiện tượng này được gọi là **Thread Pool Exhaustion** (Cạn kiệt luồng). Hãy tưởng tượng Web Server (như Tomcat) là một Phòng Giáo vụ chỉ có giới hạn 200 chuyên viên (200 worker threads). Nếu một dịch vụ phía sau (dịch vụ B) bị chậm, toàn bộ 200 chuyên viên này sẽ phải đứng chờ, dẫn đến việc các sinh viên mới (request mới) bước vào Phòng Giáo vụ sẽ không có ai phục vụ, khiến hệ thống tê liệt hoàn toàn.

Hugo Rocha trong [5] nhấn mạnh: trong distributed system, *lỗi không phải ngoại lệ — lỗi là trạng thái bình thường*. Mạng sẽ quá hạn kết nối (timeout), các dịch vụ sẽ ngưng hoạt động đột ngột (crash), và cơ sở dữ liệu sẽ phản hồi chậm. Câu hỏi không phải "nếu lỗi xảy ra" mà là "khi lỗi xảy ra".



Hình 4.5: *Cascading failure — Service B chậm làm A, D, E đều bị ảnh hưởng*

4.4.3. Bốn resilience pattern cốt lõi

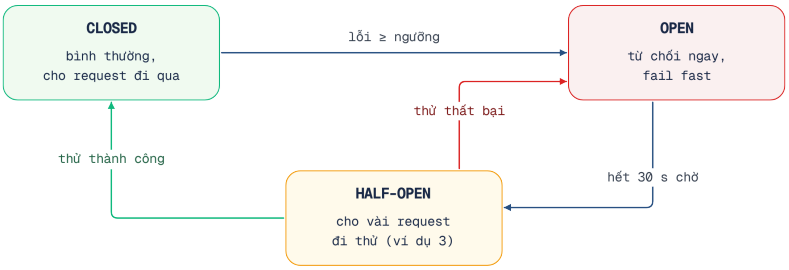
Để ngăn chặn lỗi dây chuyền (cascading failure), hệ thống phân tán áp dụng bốn cơ chế bảo vệ (resilience pattern) kinh điển: **Timeout** cắt những kết nối quá hạn để giải phóng tài nguyên; **Retry** thử lại trước lỗi mạng chập chờn tạm thời; **Circuit Breaker** ngắt mạch khi dịch vụ phía sau lỗi kéo dài; và **Bulkhead** giam rủi ro cạn kiệt tài nguyên vào những khu vực (thread pool) biệt lập để lỗi không lan rộng.

Timeout — giới hạn thời gian chờ

Timeout là tuyến phòng thủ đầu tiên và bắt buộc cho mọi lời gọi qua mạng: nếu không đặt ngưỡng chờ tối đa, một dịch vụ hạ nguồn bị treo sẽ giữ chặt luồng của bên gọi cho đến khi cạn kiệt tài nguyên. Khi cấu hình HTTP client, cần phân biệt hai loại ngưỡng: **connect timeout** (thời gian tối đa để thiết lập kết nối TCP, thường ngắn — vài trăm mili-giây đến 1 giây) và **read/response timeout** (thời gian chờ dịch vụ đích xử lý và trả dữ liệu về, đặt dựa trên phân vị độ trễ `p99` đo được). Một bất biến kiến trúc cần nhớ là *timeout ở phía gọi chỉ giải phóng luồng chờ của caller, không tự động hủy tác vụ đang chạy ở phía downstream* — nếu downstream vẫn tiếp tục ghi dữ liệu rồi bên gọi lại retry, hệ thống rất dễ sinh bản ghi trùng nếu thiếu idempotency key. Trong chuỗi gọi nhiều tầng, timeout của tầng ngoài phải lớn hơn timeout của tầng trong; nếu ngược lại, tầng ngoài bỏ cuộc trong khi tầng trong vẫn đang làm, và kết quả đó bị bỏ phí.

Circuit Breaker — chi tiết state machine

Trọng tâm của cơ chế Circuit Breaker là một mô hình trạng thái (state machine), liên tục quan sát và chuyển đổi để quyết định có nên cho phép một yêu cầu mạng được gửi đi hay không.



Cấu hình Resilience4j (Listing 4.3)

`slidingWindowSize` = 10 | `failureRateThreshold` = 50% | `waitDurationInOpenState` = 30s | `permittedNumberOfCallsInHalfOpenState` = 3

KBM: @CircuitBreaker trên JudgeCallService khi Core gọi Judge (Listing 4.2)

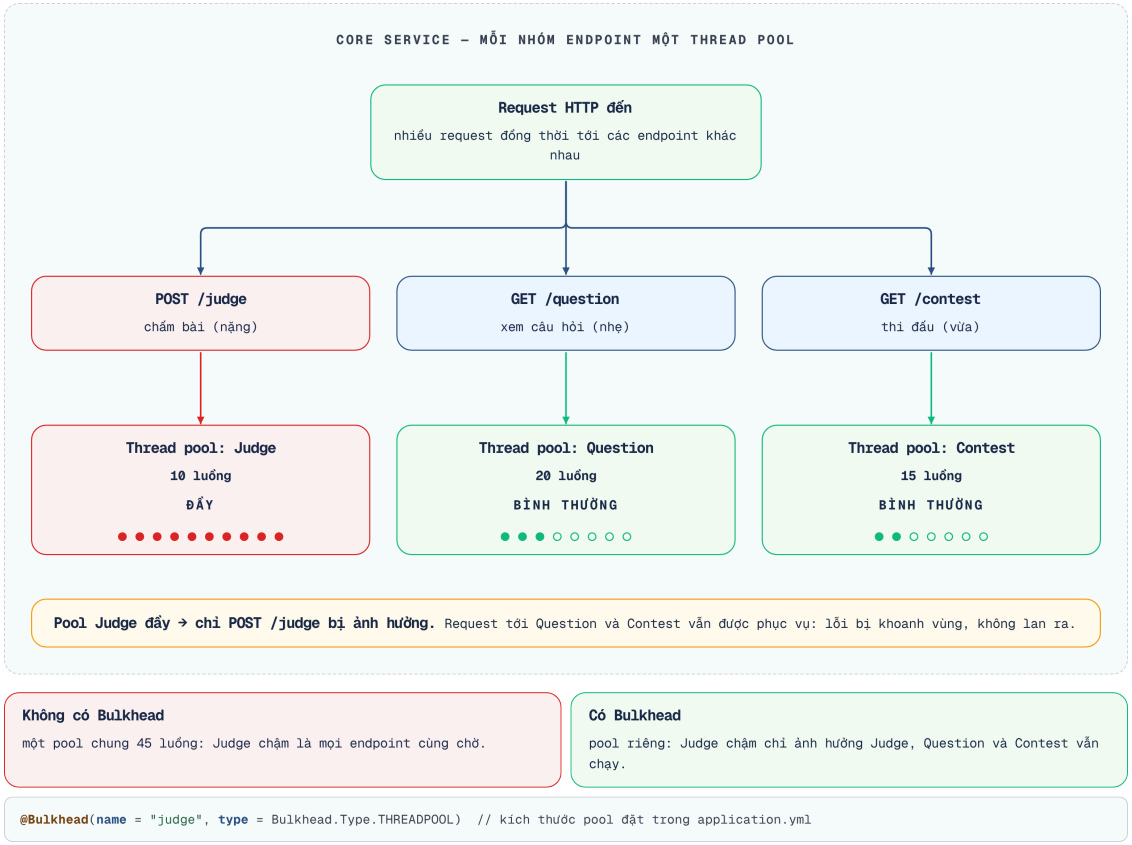
Hình 4.6: *Circuit Breaker state machine — Closed, Open, Half-Open*

Khi mạch ở trạng thái **Open**, mọi yêu cầu bị từ chối (reject) *ngay lập tức* và không được gửi tới dịch vụ đích (downstream). Cơ chế này bảo vệ cả hệ thống gọi (caller) khỏi tình trạng bị chặn luồng (thread blocking), và hệ thống được gọi (callee) khỏi sự quá tải (overwhelm) trong quá trình phục hồi (recovery).

Nhưng làm sao biết khi nào hệ thống đã phục hồi? Đó là lúc trạng thái **Half-Open** (mở một phần) phát huy tác dụng. Sau một khoảng thời gian chờ, Circuit Breaker sẽ chuyển sang Half-Open và cho phép một số lượng request giới hạn (ví dụ: 3 request) lọt qua để "thử nghiệm lưu lượng". Nếu các request này thành công, Circuit Breaker tự động đóng lại (Closed) và hệ thống hoạt động bình thường. Nếu thất bại, nó lập tức ngắt lại (Open) để tiếp tục chờ đợi.

Bulkhead — cách ly tài nguyên

Cơ chế tiếp theo trong hệ thống bảo vệ là Bulkhead, chuyên tập trung vào việc cô lập phạm vi thiệt hại khi một dịch vụ trong hệ thống bắt đầu bộc lộ sự chậm trễ bất thường. Nó thực hiện việc cô lập này bằng cách phân chia tài nguyên hệ thống — cụ thể là cấp phát các thread pool hoặc giới hạn quota (số lượng request đồng thời) riêng biệt cho từng nhóm nghiệp vụ hoặc từng API gọi ra ngoài. Thay vì sử dụng chung một pool luồng khổng lồ cho mọi request, Bulkhead đảm bảo rằng nếu một dịch vụ phản hồi chậm và chiếm dụng toàn bộ các luồng xử lý (hiện tượng Thread pool exhaustion), thì rủi ro cục bộ này sẽ chỉ bị khoanh vùng trong chính thread pool được cấp cho nó. Các pool khác vẫn còn nguyên tài nguyên để phục vụ những chức năng không liên quan — lỗi được khoanh vùng thay vì lan ra toàn hệ thống.



Hình 4.7: Bulkhead pattern — thread pool cách ly theo từng chức năng

Tên "Bulkhead" lấy từ thiết kế tàu thủy: khoang tàu bị nước tràn vào, các khoang khác vẫn khô — tàu không chìm [5, Ch.7]. Michael Nygard giới thiệu pattern này trong *Release It!* (2007).

Retry — thử lại có kiểm soát

Chỉ thử lại (retry) các thao tác bảo đảm idempotency (GET, PUT, DELETE). Việc thử lại các thao tác POST có thể tạo ra dữ liệu trùng lặp (duplicate data). Muốn retry POST thì request phải mang idempotency key để bên nhận nhận ra và bỏ qua lần gửi lặp. Ngoài ra, không bao giờ thử lại tức thì liên tiếp: phải giãn cách thời gian chờ theo cấp số nhân (**exponential backoff**, ví dụ 200ms → 400ms → 800ms) kết hợp độ lệch ngẫu nhiên (**jitter**) để hàng trăm client không đồng loạt dội lại yêu cầu vào đúng một thời điểm (hiện tượng *retry storm* / *thundering herd*).

Kết hợp các pattern

Trong thực tế, các pattern thường kết hợp theo thứ tự:

```
Request → Retry (3 lần, backoff) → Circuit Breaker → Timeout (5s)
        → Bulkhead → Actual Call
```

Chuỗi kết hợp trên tạo ra một quy trình kiểm soát lỗi đa tầng. Khi một yêu cầu mạng được gửi đi, nó phải vượt qua từng chốt kiểm tra này trước khi thực sự chạm đến hệ thống đích.

Spring Cloud Circuit Breaker với Resilience4j là lựa chọn phổ biến cho Java microservices: logic retry và circuit breaker được khai báo bằng annotation và cấu hình, không phải viết tay.

4.4.4. Triển khai thực tế với Resilience4j — mã ví dụ cho LMS

Đoạn mã dưới đây minh họa cách áp dụng cho Core Service của KBM — đây là đích của Giai đoạn 3 trong lộ trình Ch.10, chưa phải mã đang chạy. Spring Cloud Circuit Breaker kết hợp cùng Resilience4j cung cấp một bộ công cụ cho phép tích hợp cả bốn cơ chế bảo vệ thông qua các annotation. Cách tiếp cận này tách biệt logic nghiệp vụ khỏi logic xử lý lỗi và quản lý độ tin cậy.

Listing 4.2: *Resilience4j annotations cho Judge Feign Client*

```
// Core Service — gọi Judge với đầy đủ các resilience pattern
@Service
public class JudgeCallService {
    private static final Logger log =
            LoggerFactory.getLogger(JudgeCallService.class);
    private final JudgeClient judgeClient;
    private final JudgeRetryQueue judgeRetryQueue;

    public JudgeCallService(JudgeClient judgeClient,
                            JudgeRetryQueue judgeRetryQueue) {
        this.judgeClient = judgeClient;
        this.judgeRetryQueue = judgeRetryQueue;
    }

    // Thứ tự khai báo annotation không ảnh hưởng đến luồng thực thi (execution)
    // — Spring AOP sử dụng @Order của aspect
    // Thứ tự thực thi thực tế:
    // Retry → CircuitBreaker → TimeLimiter → Bulkhead → call
    // Retry an toàn dù đây là POST: JudgeRequest mang submissionId
    // làm idempotency key (§4.4)
    @Retry(name = "judge", fallbackMethod = "judgeFallback")
    @CircuitBreaker(name = "judge")
    @TimeLimiter(name = "judge")
    @Bulkhead(name = "judge")
    public CompletableFuture<JudgeResult> executeSQL(JudgeRequest request) {
        return CompletableFuture.supplyAsync(() →
            judgeClient.submitSql(request)
        );
    }

    // Phương thức dự phòng (Fallback) khi mạch bị ngắt (OPEN)
    // hoặc vượt quá số lần thử lại (retries)
    private CompletableFuture<JudgeResult> judgeFallback(
            JudgeRequest request, Throwable t) {
        if (t instanceof BusinessException) {
            // Lỗi nghiệp vụ: không đưa vào hàng đợi
            return CompletableFuture.failedFuture(t);
        }
        log.warn("Judge unavailable: {}, submissionId={}",
                 t.getMessage(), request.getSubmissionId());
        // Phải ghi command vào durable queue/outbox TRƯỚC khi hứa "sẽ chấm sau"
        // (idempotency key = submissionId); nếu enqueue thất bại thì trả lỗi
        // tạm thời (503 + Retry-After) thay vì một lời hứa không ai thực hiện
        judgeRetryQueue.enqueue(request);
        return CompletableFuture.completedFuture(
            JudgeResult.pending("Sandbox đang bận, bài sẽ được chấm khi sẵn sàng")
        );
    }
}
```

Về phân tầng Circuit Breaker: instance `judge` trong ví dụ trên bảo vệ Core khỏi *cụm Judge như một tổng thể* — đúng vai của Core, vì Core chỉ nói chuyện với bộ điều phối `kbm-judge-sql` (Giai đoạn 1, §4.5). Còn lớp bảo vệ chi tiết theo từng DBMS worker (`kbm-sandbox-mysql`, `kbm-sandbox-sqlserver`...) thuộc về chính coordinator: mỗi HTTP client tới một worker cần Circuit Breaker riêng (Giai đoạn 2, §4.5), để `kbm-sandbox-mysql` quá tải không kéo sập đường chấm của `kbm-sandbox-sqlserver`. Hai tầng này bổ trợ nhau, không thay thế nhau.

Đoạn mã trên minh họa cách phối hợp nhiều lớp bảo vệ trên cùng một phương thức khai báo, tạo thành một cơ chế phòng vệ trước các rủi ro sụp đổ dây chuyền. Dù vậy, bao bọc các lời gọi mạng chậm bằng `CompletableFuture` có thể làm cạn kiệt tài nguyên luồng, đặc biệt nếu lập trình viên không nắm rõ cơ chế cấp phát thread pool mặc định bên dưới.

⚠ Cấp Executor cho CompletableFuture

Đoạn `supplyAsync(...)` ở trên dùng overload *một tham số*, nên task chạy trên **common ForkJoinPool** dùng chung toàn JVM. Khi kết hợp với `@TimeLimiter` và `@Bulkhead`, các lệnh gọi kéo dài (chờ phản hồi từ Judge) có thể làm cạn kiệt bể luồng dùng chung (thread starvation), gây đình trệ toàn bộ các tác vụ khác trong hệ thống — vì mọi `CompletableFuture` không chỉ định executor và cả các parallel stream trong cùng JVM đều xếp hàng chờ trên chính pool này. Trên production, nên truyền một `Executor` riêng (`supplyAsync(() → ..., judgeExecutor)`) hoặc dùng `@Bulkhead(type = Bulkhead.Type.THREADPOOL)` để Resilience4j quản lý thread pool tách biệt cho nhóm call này (khi đó gọi `judgeClient.submitSql(request)` ngay trong method rồi trả `CompletableFuture.completedFuture(result)`, không gọi `supplyAsync` nữa). Đồng thời, `@TimeLimiter` chỉ hủy chờ trên `Future` chứ không tự đóng socket HTTP đang bị chặn bên dưới — bắt buộc phải cấu hình kèm `connectTimeout` và `readTimeout` cho chính OpenFeign client (`spring.cloud.openfeign.client.config.<name>.*`, xem Listing 4.3).

Từ Java 21 LTS, **Virtual Threads** (Project Loom) thay đổi đáng kể bài toán khan hiếm luồng ở trên: mỗi request có thể chạy trên một luồng ảo gần như "miễn phí" (JVM quản lý hàng triệu luồng ảo trên vài luồng hệ điều hành), nên phần lớn các lệnh gọi blocking I/O không còn chiếm giữ luồng nền tảng đắt đỏ (trên Java 21, luồng ảo vẫn bị ghim khi chặn trong khối `synchronized`; JDK 24 mới gỡ hạn chế này). Spring Boot 3.2+ chỉ cần bật `spring.threads.virtual.enabled=true` để áp dụng cho mô hình thread-per-request. Tuy nhiên, virtual threads chỉ giải quyết chi phí *chờ đợi I/O* — chúng không thay thế được Timeout, Retry, Circuit Breaker hay Bulkhead, vì dịch vụ phía sau chậm thì vẫn chậm; các resilience pattern trong chương này vẫn nguyên giá trị.

Ngoài khai báo annotation trong mã nguồn, Resilience4j cho phép tinh chỉnh các ngưỡng bảo vệ qua tệp cấu hình bên ngoài. Điều này mang lại lợi thế quản trị: các ngưỡng bảo vệ được tách khỏi mã nguồn, thay đổi không cần biên dịch lại — thông thường chỉ cần khởi động lại service để nạp giá trị mới. Muốn thay đổi "nóng" thật sự lúc runtime, cần bổ sung cơ chế chuyên dụng như Spring Cloud Config kết hợp `@RefreshScope`/actuator refresh endpoint.

Listing 4.3: *Cấu hình Resilience4j (YAML) cho LMS*

```yaml
# application.yml — Resilience4j configuration
resilience4j:
  circuitbreaker:
    instances:
      judge:
        slidingWindowSize: 10              # Đánh giá trên 10 requests gần nhất
```

```
        failureRateThreshold: 50           # Mở circuit khi 50% requests lỗi
        waitDurationInOpenState: 30s       # Chờ 30s trước khi thử lại (half-open)
        permittedNumberOfCallsInHalfOpenState: 3  # Thử 3 calls khi half-open
        recordExceptions:                  # Chỉ count những exceptions này
          - java.io.IOException
          - java.util.concurrent.TimeoutException
          - feign.RetryableException
          - feign.FeignException$ServiceUnavailable

  retry:
    instances:
      judge:
        maxAttempts: 3                     # Tối đa 3 lần thử
        waitDuration: 1s                   # Chờ 1s giữa các lần
        enableExponentialBackoff: true     # Kích hoạt backoff
        exponentialBackoffMultiplier: 2    # 1s → 2s (3 lần thử = 2 lần chờ)
        enableRandomizedWait: true         # Thêm jitter để tránh retry storm
        randomizedWaitFactor: 0.5          # Hệ số ví dụ, cần tinh chỉnh theo tải
        retryExceptions:
          - java.io.IOException            # Chỉ retry lỗi tạm thời
          # Bắt buộc để Retry bắt được timeout từ TimeLimiter
          - java.util.concurrent.TimeoutException
          # Lỗi mạng mà Feign bọc lại — thiếu dòng này Retry sẽ bỏ qua
          - feign.RetryableException
          # 503 từ downstream cũng là lỗi tạm thời đáng retry
          - feign.FeignException$ServiceUnavailable
        ignoreExceptions:
          - com.kbm.exception.BusinessException  # KHÔNG retry lỗi logic

  timelimiter:
    instances:
      judge:
        timeoutDuration: 5s               # Timeout 5s cho mỗi lần thử (attempt)

  bulkhead:
    instances:
      judge:
        maxConcurrentCalls: 10            # Tối đa 10 calls đồng thời đến Judge
        maxWaitDuration: 500ms            # Chờ tối đa 500ms nếu bulkhead đầy

spring:
  cloud:
    openfeign:
      client:
        config:
          kbm-judge-sql:
            connectTimeout: 1000          # 1s thiết lập kết nối TCP
            # connectTimeout + readTimeout = 4,5s, nhỏ hơn timeoutDuration (5s)
            readTimeout: 3500
```

Ngưỡng 5 giây ở đây chỉ hợp với lượt chấm truy vấn đơn giản trên đường đồng bộ; các lượt chấm kéo dài 5–30 giây (§4.1) đi đường bất đồng bộ ở Giai đoạn 4 (§4.5), không đi qua lời gọi này.

Các cấu hình YAML phía trên đã thiết lập các ngưỡng an toàn cho quá trình tương tác giữa hệ thống Core và hệ thống chấm điểm Judge. Tuy nhiên, các tham số này không phải giá trị tĩnh dùng chung cho mọi hệ thống. Tùy theo môi trường và mức tải dự kiến, kỹ sư cần giám sát và tinh chỉnh liên tục để đạt điểm cân bằng: đủ nhạy để ngắt mạch khi có lỗi thật, nhưng đủ bao dung

với những chập chờn mạng tạm thời. Bảng 4.5 dưới đây cung cấp một số giá trị tham khảo cốt lõi được khuyến nghị cho các kịch bản triển khai khác nhau.

Bảng 4.5: *Giá trị khuyến nghị cho từng môi trường*

Tham số	Phát triển	Vận hành (LMS)	Tải cao
`slidingWindowSize`	5	10	20
`failureRateThreshold`	50%	50%	30%
`waitDurationInOpenState`	10s	30s	60s
`retry.maxAttempts`	1	3	3
`timelimiter.timeoutDuration`	10s	5s	3s
`bulkhead.maxConcurrentCalls`	5	10	25

Bên cạnh các thông số ngưỡng, còn một yếu tố thường bị cấu hình sai và gây ra lỗi khó phát hiện: thứ tự áp dụng các annotation decorator trên cùng một phương thức. Theo tài liệu Resilience4j, các decorator được lồng theo thứ tự mặc định **từ ngoài vào trong**: `Retry( CircuitBreaker( TimeLimiter( Bulkhead( call ))))`. Nghĩa là Retry nằm ở lớp ngoài cùng — mỗi lần retry sẽ chạy lại toàn bộ chuỗi CircuitBreaker → TimeLimiter → Bulkhead. Thứ tự này quan trọng: nếu TimeLimiter bao bọc bên ngoài Retry, một timeout exception đầu tiên sẽ hủy bỏ toàn bộ chuỗi thử lại, làm vô hiệu hóa hoàn toàn mục đích của cơ chế Retry. Cấu hình mặc định của Resilience4j đã xử lý đúng thứ tự này. Hơn nữa, vì `TimeLimiter` ném ra `TimeoutException` khi hết hạn, chúng ta **bắt buộc phải khai báo** `java.util.concurrent.TimeoutException` trong danh sách `retryExceptions` của Retry — nếu thiếu, khi timeout xảy ra, Retry sẽ không kích hoạt và báo lỗi ngay lập tức. Cũng vì Retry bọc ngoài TimeLimiter, tổng thời gian chờ tối đa mà bên gọi có thể phải chịu bằng tổng timeout của các lần thử cộng thời gian backoff giữa chúng (ví dụ với `maxAttempts: 3`, `timeoutDuration: 5s` và backoff `1s → 2s`, trường hợp xấu nhất có thể kéo dài khoảng `3 × 5s + 1s + 2s ≈ 18s` (jitter làm thời gian chờ dao động quanh mức này)); do đó cần đặt `timeoutDuration` của mỗi lần thử đủ ngắn hoặc giới hạn tổng thời gian chờ ở tầng gọi.

Design for Failure

Trong hệ thống phân tán, lỗi không phải là ngoại lệ — lỗi là trạng thái bình thường. Hãy thiết kế mọi lời gọi liên dịch vụ với giả định rằng nó sẽ thất bại. Timeout, Retry, Circuit Breaker, Bulkhead — đây không phải những lựa chọn tùy ý, mà là điều kiện tiên quyết.

— Tổng hợp từ Hugo Rocha [5, Ch.7] và Michael Nygard, Release It!

4.5. Case study: KBM

Để hiểu rõ sự phối hợp giữa các thành phần, hãy xem xét hệ thống chấm điểm SQL của KBM như một hội đồng thi. Trong đó, Coordinator hoạt động như thư ký hội đồng điều phối đề thi, còn các worker DBMS là các giảng viên chuyên trách từng môn học khác nhau.

4.5.1. Bài toán

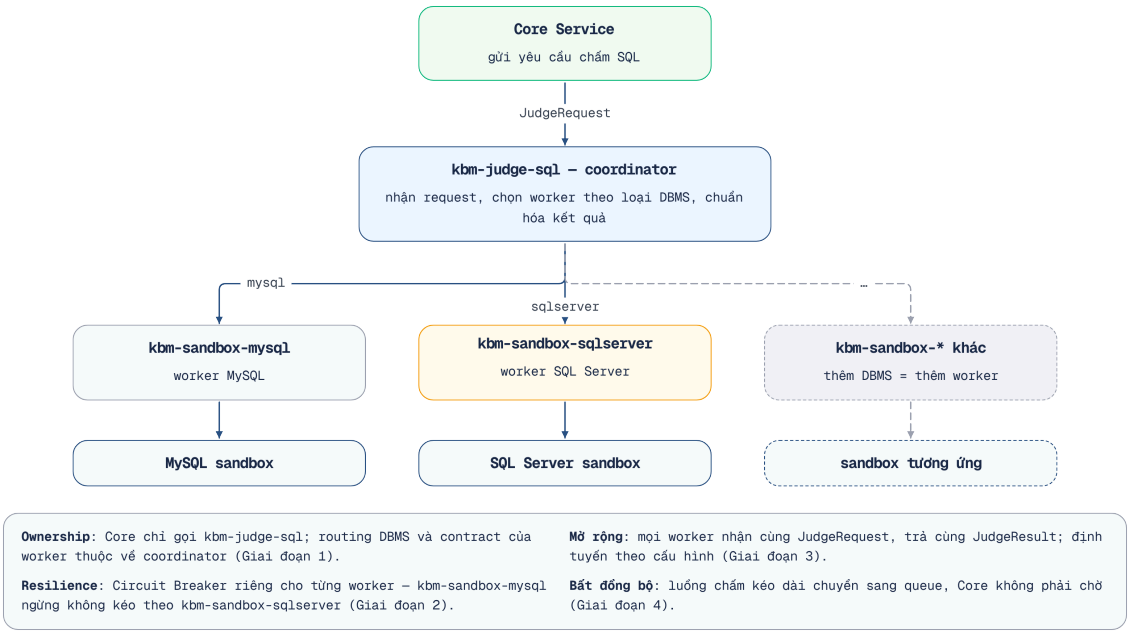
KBM hỗ trợ sinh viên thực hành SQL trên nhiều DBMS. Mỗi DBMS chạy trong sandbox container riêng biệt. Khi sinh viên nộp bài, hệ thống cần:

1. Xác định DBMS nào (dựa vào câu hỏi)
2. Gửi SQL đến đúng sandbox service
3. So sánh kết quả với đáp án (SHA-256 hash)

4. Trả về kết quả

4.5.2. Hiện trạng: Strategy Pattern + OpenFeign

Với yêu cầu hỗ trợ đa dạng các hệ quản trị cơ sở dữ liệu (DBMS), kiến trúc hiện tại của hệ thống KBM sử dụng mẫu thiết kế Strategy kết hợp cùng OpenFeign. Trong mô hình phối hợp này, một dịch vụ điều phối trung tâm `kbm-judge-sql` được xây dựng để hoạt động như một thư ký hội đồng, chịu trách nhiệm tiếp nhận toàn bộ các yêu cầu chấm bài từ hệ thống Core. Sau đó, dựa trên loại cơ sở dữ liệu được chỉ định, nó sẽ phân tích và điều hướng (dispatch) các đoạn mã SQL này đến đúng các worker chuyên trách tương ứng, đảm bảo sự phân tách trách nhiệm rõ ràng.



Hình 4.8: *Bộ điều phối `kbm-judge-sql` — điều hướng SQL đến các worker `kbm-sandbox-*` theo DBMS*

Core Service không nên biết chi tiết MySQL hay SQL Server được xử lý ở worker nào. Nó gửi yêu cầu chấm đến `kbm-judge-sql`; `kbm-judge-sql` làm nhiệm vụ coordinator: nhận database type, chọn worker tương ứng (`kbm-sandbox-mysql`, `kbm-sandbox-sqlserver`, ...), gọi worker qua HTTP/Feign hoặc queue, rồi chuẩn hóa kết quả trả về Core. Đây là pattern phổ biến: một coordinator giữ contract nghiệp vụ ổn định, các worker phía sau chuyên môn hóa theo runtime/DBMS.

4.5.3. Phân tích các vấn đề

Mặc dù kiến trúc hiện hành với thiết kế Strategy Pattern đã giúp tách biệt các trình điều khiển cơ sở dữ liệu một cách trực quan, nó vẫn bộc lộ các điểm yếu khi số worker và lượng bài nộp tăng, hoặc khi gặp lỗi mạng. Bảng 4.6 dưới đây trình bày chi tiết năm vấn đề cốt lõi của thiết kế hiện tại cùng với các phương pháp giải quyết tối ưu.

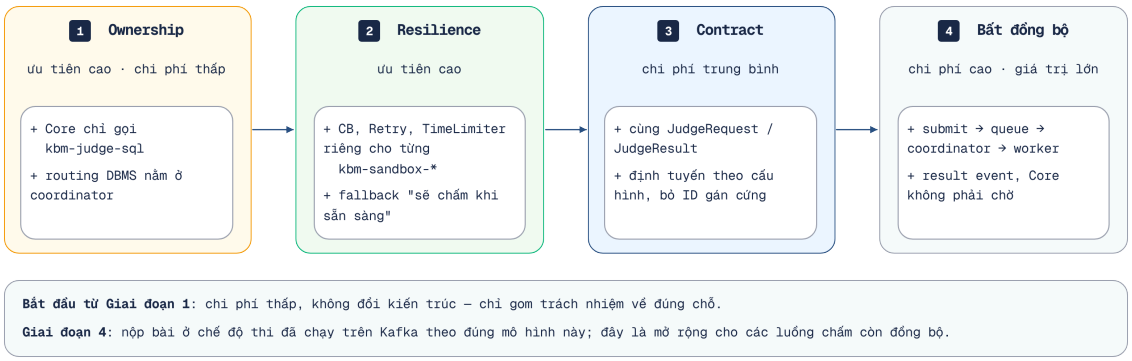
Bảng 4.6: *Phân tích vấn đề SQL Judge coordinator/worker*

#	Vấn đề	Mô tả	Giải pháp khuyến nghị
1	**Routing ownership chưa rõ**	Nếu Core cũng chứa logic chọn DBMS, routing bị rò rỉ ra ngoài Judge boundary	Single ownership: `kbm-judge-sql` là coordinator duy nhất

#	Vấn đề	Mô tả	Giải pháp khuyến nghị
2	**Định tuyến cứng (hardcoded routing) dựa trên các ID bí ẩn (Magic ID)**	Database type xác định bằng hardcoded UUID/string	Enum hoặc configuration-driven registry
3	**Tính chống chịu (resilience) theo từng worker chưa rõ ràng**	`kbm-sandbox-mysql` ngừng hoạt động không được phép làm gián đoạn toàn bộ dịch vụ `kbm-judge-sql` hay các worker khác	Circuit Breaker/Timeout riêng cho từng worker
4	**Cơ chế dự phòng (fallback) chưa nhất quán**	Worker của DBMS gặp sự cố → người dùng nhận lỗi kỹ thuật hoặc tiến trình nộp bài bị kẹt (stuck)	Phản hồi dự phòng (fallback), thử lại/hàng đợi, trạng thái rõ ràng
5	**Contract của worker chưa được chuẩn hóa**	Các `kbm-sandbox-*` dễ trả về định dạng/kết quả (format/result) khác nhau	Chuẩn hóa `JudgeRequest`/`JudgeResult` contract

4.5.4. Lộ trình chuyển đổi đề xuất

Để giải quyết các rủi ro thiết kế đã phân tích và chuẩn bị cho sự gia tăng khối lượng tải, hệ thống chấm điểm SQL cần một lộ trình tái cấu trúc gồm bốn giai đoạn. Lộ trình này bắt đầu từ việc củng cố sự ổn định cục bộ thông qua các mẫu thiết kế bảo vệ (resilience), cho đến việc thay đổi mô hình tương tác sang dạng bất đồng bộ hướng sự kiện (event-driven).



Hình 4.9: Lộ trình chuyển đổi cho SQL Judge — từ resilience đến bất đồng bộ

- **Giai đoạn 1 — Khóa ownership**: Ưu tiên cao, chi phí triển khai thấp. Giữ nguyên tắc Core chỉ gọi `kbm-judge-sql`; toàn bộ logic định tuyến (routing) hệ quản trị cơ sở dữ liệu phải được quản lý bởi bộ điều phối `kbm-judge-sql`. Không để logic định tuyến hệ quản trị cơ sở dữ liệu rò rỉ sang Core hoặc các service không sở hữu nghiệp vụ chấm SQL.
- **Giai đoạn 2 — Thêm resilience theo worker**: Ưu tiên cao. `@CircuitBreaker`, `@Retry`, `@TimeLimiter` riêng cho từng `kbm-sandbox-*`. Khi `kbm-sandbox-mysql` ngừng hoạt động → hệ thống sẽ kích hoạt phản hồi dự phòng (fallback): "Sandbox MySQL đang bận, bài sẽ được chấm khi sẵn sàng", không ảnh hưởng `kbm-sandbox-sqlserver`.
- **Giai đoạn 3 — Chuẩn hóa worker contract**: Chi phí triển khai trung bình. Tất cả worker nhận cùng `JudgeRequest`, trả cùng `JudgeResult`, định tuyến dựa trên cấu hình (config-driven) thay vì gán cứng ID.
- **Giai đoạn 4 — Chuyển các luồng xử lý kéo dài sang mô hình bất đồng bộ**: Chi phí triển khai cao, giá trị lớn. Submit → Kafka/queue → `kbm-judge-sql` → `kbm-sandbox-*` worker → Result event → Core cập nhật. Người dùng không phải chờ đồng bộ — sẽ nhận được thông báo (notification) khi có kết quả. Luồng nộp bài chế độ thi SQL đã chạy trên Kafka theo đúng mô hình này

(Chương 5); Giai đoạn 4 là việc mở rộng nó cho các luồng chấm kéo dài còn lại đang đi đường đồng bộ.

4.5.5. Tích hợp ngoài: QLĐT, GitHub Classroom và Network Practice

Không phải mọi tích hợp trong KBM đều nên đi qua cùng một protocol. Với QLĐT hoặc GitHub Classroom, KBM nên đặt **Anti-Corruption Layer** ở boundary: adapter dịch các contract bên ngoài thành lệnh/sự kiện nội bộ, che giấu định dạng cấu trúc dữ liệu, các lỗi và giới hạn lưu lượng (rate limit) của hệ thống đối tác. Với Network Practice, điểm đáng học là ngược lại: context này được thiết kế độc lập để đánh giá giao tiếp mạng, không phụ thuộc vào service chấm SQL. Vì vậy, nó là ví dụ tốt cho bounded context theo protocol, còn vấn đề coordinator/worker cần phân tích nằm ở SQL Judge.

ⓘ Xây hay mua --- tự viết hay tích hợp DOMjudge cho bộ chấm code

Cùng tinh thần Anti-Corruption Layer, module chấm code đa ngôn ngữ (`kbm-judge-code`) minh họa một quyết định **'xây hay mua' (build-vs-buy)** điển hình. Thế hệ đầu là bộ chấm đồng bộ *tự viết* (C/C++ và Java) — kiểm soát tối đa nhưng nhóm phát triển phải tự đảm đương và duy trì toàn bộ trình biên dịch (compiler), môi trường thực thi (runner) cũng như các cơ chế bảo mật. Thế hệ hiện tại thay bằng **DOMjudge** — một dịch vụ OSS đã được kiểm chứng qua nhiều kỳ thi ICPC. Bằng việc chấp nhận một thành phần phụ thuộc (dependency) bên ngoài, KBM loại bỏ cả một lớp rủi ro tự xây (sandbox, đa ngôn ngữ, bảo trì compiler). Bài học sync communication: với một service đồng bộ phức tạp và nhạy cảm bảo mật, *tích hợp một giải pháp đã kiểm chứng* thường thắng *tự duy trì* — miễn đặt adapter rõ ràng ở boundary để cô lập contract ngoài. (Khía cạnh cô lập/sandbox bảo mật: Ch.9.)

Trên thực tế, nhiều hệ thống bắt đầu với sync (đơn giản, dễ debug) rồi chuyển dần sang async khi cần mở rộng. Richardson trong [2a] minh họa quá trình này: ban đầu là các lệnh gọi REST giữa các dịch vụ, sau đó chuyển sang mẫu thiết kế Saga khi phát sinh nhu cầu giao dịch phân tán (distributed transaction). Hệ thống KBM đang ở giữa tiến trình chuyển đổi này: một số luồng xử lý (flow) đã chuyển sang mô hình bất đồng bộ (sử dụng Kafka để nộp bài), một số luồng vẫn duy trì đồng bộ (sử dụng Feign để truy vấn), và phân hệ SQL Judge cần được tách bạch rõ ràng giữa coordinator và worker trước khi tiến hành mở rộng theo chiều ngang (horizontal scaling) một cách an toàn. Chương 5 sẽ tiếp tục phần async.

⚠ Sai lầm thường gặp khi dùng giao tiếp đồng bộ

Khi triển khai các cuộc gọi API đồng bộ, các nhóm phát triển thường mắc phải một số lỗi kinh điển dẫn đến sự sụp đổ dây chuyền của hệ thống (cascading failure). Nhận diện và phòng tránh những cái bẫy này là nền tảng để xây dựng hệ thống bền bỉ.

- **Không đặt timeout cho inter-service calls (Tin vào timeout mặc định)**: Nhiều HTTP client (kể cả Feign nếu không cấu hình) có timeout mặc định rất dài hoặc chờ vô hạn. Giống như việc sinh viên đứng chờ mãi ở phòng giáo vụ mà không biết người phụ trách đã nghỉ ốm. Hậu quả: khi dịch vụ phía sau (downstream) phản hồi chậm hoặc bị treo, luồng xử lý của bên gọi bị giữ lại vô thời hạn → dẫn đến cạn kiệt thread pool → gây lỗi dây chuyền (cascading failure). *Phòng tránh*: Luôn cấu hình thời gian chờ kết nối/đọc (connect/read timeout) một cách tường minh (1–3 giây là điểm khởi đầu hợp lý cho lời gọi nội bộ

— tham chiếu default 1s của Hystrix; con số cuối cùng phải lấy từ p99 đo được của chính downstream).

- **Tin rằng mạng luôn ổn định**: Viết code như thể mọi HTTP call đều thành công. Hậu quả: lỗi đầu tiên trên production mới phát hiện không có retry, không fallback. *Phòng tránh*: Áp dụng các resilience pattern từ đầu (Circuit Breaker + Retry + Timeout) — ngay cả khi chưa cần scale (§4.4).
- **Thử lại mà không có độ trễ lùi dần (backoff) và không kiểm tra idempotency**: Thử lại ngay lập tức một thao tác POST (có thể tạo ra dữ liệu trùng lặp, duplicate data). Giống như sinh viên bấm nút nộp bài thi liên tục khi mạng chậm, dẫn đến nộp 10 bản sao. Hậu quả: vừa khuếch đại tải lên dịch vụ đang quá tải/suy yếu (tạo retry storm), dữ liệu sai, người dùng bị tính điểm trùng. *Phòng tránh*: Chỉ retry thao tác idempotent (GET, PUT, DELETE) hoặc dùng idempotency key cho POST, dùng exponential backoff + jitter, giới hạn số lần.
- **Circuit Breaker không có cơ chế dự phòng (fallback) hợp lý**: Ngắt mạch (open circuit) nhưng vẫn trả về unhandled exception cho người dùng. *Phòng tránh*: Định nghĩa fallback trả về giá trị mặc định/cache hoặc thông báo "Hệ thống đang bận, sẽ xử lý sau" để đảm bảo hệ thống suy giảm chất lượng một cách có kiểm soát (graceful degradation).
- **Chuỗi gọi đồng bộ quá dài**: A→B→C→D đồng bộ. Hậu quả: độ trễ cộng dồn, và sự chậm trễ của một mắt xích sẽ làm đình trệ toàn bộ chuỗi giao tiếp — chính là distributed monolith. Việc bắt một sinh viên phải tự chạy qua 4 phòng ban khác nhau để xin đủ 4 chữ ký trong cùng một buổi sáng là không khả thi. *Phòng tránh*: Rút ngắn chuỗi, cân nhắc async (Chương 5) cho các bước không cần kết quả ngay.

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

- **Service Discovery**: service-discovery.html
- **Circuit Breaker**: circuit-breaker.html
- **Load Shedding**: load-shedding.html

4.6. Tổng kết

Giao tiếp đồng bộ là mô hình đơn giản nhất nhưng tiềm ẩn rủi ro lớn nhất trong hệ thống phân tán (distributed system). Temporal coupling, cascading failure, và độ trễ cộng dồn là ba vấn đề cốt lõi mà các kỹ sư phần mềm phải đối mặt. Chọn kiểu tương tác và giao thức theo từng ranh giới: REST cho API hướng người dùng, gRPC khi giao tiếp nội bộ cần thông lượng cao hoặc streaming. Bài toán SQL Judge của KBM cho thấy một coordinator giữ contract ổn định giúp các worker phía sau thay đổi độc lập.

OpenFeign đơn giản hóa các lệnh gọi liên dịch vụ (service-to-service call) thành các khai báo giao diện Java (Java interface declaration), loại bỏ các đoạn mã HTTP rườm rà (boilerplate HTTP code). Service Discovery (Eureka) giải quyết bài toán "tìm dịch vụ ở đâu" — một vấn đề không tồn tại trong monolith nhưng đặc biệt quan trọng trong microservices.

Các resilience pattern — Timeout, Retry, Circuit Breaker, Bulkhead — không phải là các tùy chọn có hay không cũng được. Chúng là điều kiện tiên quyết để hệ thống microservices hoạt động ổn

định trên production. Phân tích SQL Judge của KBM cho thấy resilience theo từng worker và cơ chế dự phòng là những điểm cần củng cố trước khi mở rộng (§4.5).

Ở Chương 5, chúng ta sẽ khám phá **giao tiếp bất đồng bộ** — giải pháp cho những giới hạn của giao tiếp đồng bộ (sync): Apache Kafka, kiến trúc hướng sự kiện (event-driven architecture), và cách KBM ứng dụng cơ chế truyền tin (messaging) cho luồng xử lý nộp bài (submission pipeline).

4.7. Đọc thêm

Sách tham khảo chính:

1. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.3: Interprocess Communication, Service Discovery
2. [4a] Sam Newman, *Building Microservices* — Ch.4: Integration, Smart Endpoints and Dumb Pipes
3. [5] Hugo Rocha, *Practical Event-Driven Microservices Architecture* — Ch.7: Resilience & Reliability

Sách bổ trợ:

4. [2b] Chris Richardson, *Microservices Patterns*, 2nd Ed. — Ch.4: Coupling Taxonomy (design-time, build-time, runtime, infrastructure), Iceberg Principle, DRY in distributed systems; Ch.8: IPC patterns
5. Michael Nygard, *Release It!*, 2nd Ed. (2018) — Circuit Breaker, Bulkhead, Timeout patterns

Nguồn trực tuyến:

- [W7] Netflix Technology Blog — “Making the Netflix API More Resilient” (2011)
- Resilience4j documentation — resilience4j.readme.io

Bài tập: 4-0, 4-1, 4-2 ở phần Bài tập & case study cuối sách.

Tài liệu tham khảo

[2a] Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.

[2b] Richardson, C. (2025). *Microservices Patterns* (2nd ed.; bản MEAP, truy cập 07/2026). Manning Publications.

[4a] Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O’Reilly Media.

[5] Rocha, H. F. O. (2022). *Practical Event-Driven Microservices Architecture: Building Sustainable and Highly Scalable Event-Driven Microservices*. Apress.